

La Regla de Oro del Alerting

Una alerta válida debe cumplir los TRES criterios simultáneamente:

Important

Si la respuesta a **cualquiera** de los tres es **NO** → no es una alerta.
Es ruido, y el ruido es el enemigo de la confiabilidad.

Monitoring &

Observability

Lucas Bonanni

Monitoring & Observability

Introduction

El monitoreo y análisis se refiere a la práctica de **observar, rastrear y analizar** el rendimiento y la salud de los sistemas y aplicaciones.

Involucra la recolección de datos, métricas y logs para ganar insights y hacer mejoras informadas.

Es crucial para identificar y resolver problemas de manera **proactiva**, asegurando la optimización continua de la operación y el rendimiento.

Note



"You can't manage what you can't measure." — Peter Drucker

SLI / SLO / SLA

El problema sin objetivos

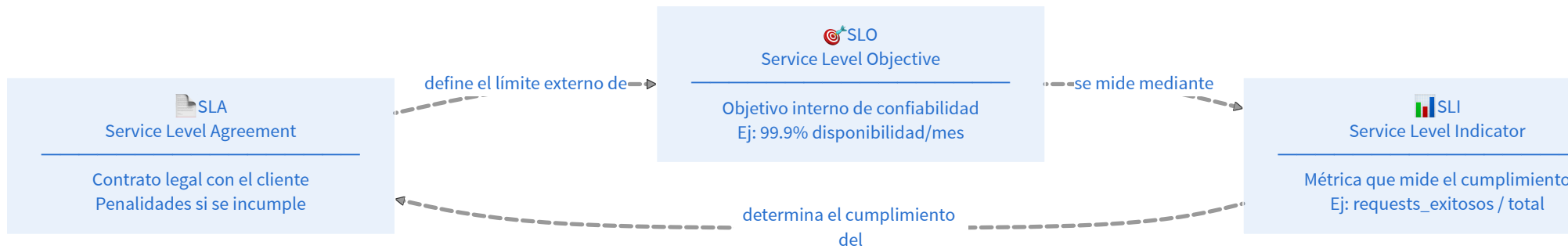
Principio fundamental

El monitoreo sin objetivos de nivel de servicio es solo ruido en una pantalla.

Sin SLOs, no sabes qué métricas importan, cuándo alertar, ni si tu sistema está bien o mal.

Primero define **qué garantizas**. Luego decide **qué medir**.

¿Qué son SLI, SLO y SLA?



SLI / SLO / SLA — Definiciones

Concepto	Definición	Ejemplo concreto
SLI	Métrica cuantitativa que mide el comportamiento real del servicio	<code>requests_exitosas / total_requests = 99.95%</code>
SLO	Objetivo de confiabilidad que debe cumplir el SLI en un período	<code>SLI >= 99.9%</code> medido en ventana de 30 días
SLA	Acuerdo contractual con penalidades si el SLO se incumple	Crédito del 10% si disponibilidad < 99.5% en el mes

Error común en equipos

Monitorear todo sin definir primero qué nivel de servicio se quiere garantizar.
Define primero tus SLOs → luego elige qué SLIs capturar → luego instrumenta.

Error Budget

¿Qué es el Error Budget?

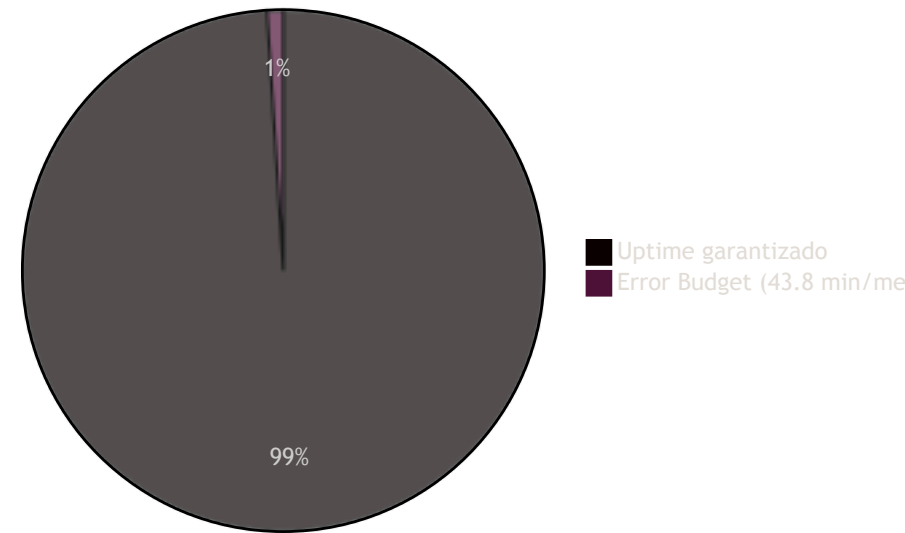
Es la cantidad de tiempo/requests que **pueden fallar** sin romper el SLO.

- SLO: **99.9%** de disponibilidad mensual
- Error Budget: **0.1% = 43.8 minutos/mes**

¿Para qué sirve?

- ¿Puedo hacer un deploy hoy?
- ¿Es seguro este experimento?
- ¿Necesitamos frenar features y pagar deuda técnica?

Error Budget mensual – SLO 99.900%

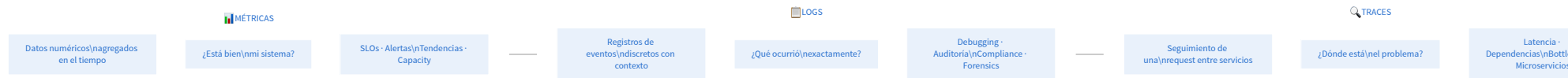


Tip

El Error Budget convierte conversaciones subjetivas (“¿deployamos?”) en decisiones basadas en datos.

Los 3 Pilares de la Observabilidad

Los 3 Pilares



Métricas — ¿Para qué sirven?

Métricas: Datos numéricos agregados en el tiempo. Baratas de almacenar, rápidas de consultar.

✅ Úsalas para:

- Medir si tu sistema cumple los SLOs
- Detectar degradaciones de rendimiento
- Configurar alertas accionables
- Visualizar tendencias a largo plazo
- Capacity planning

❌ No las uses para:

- Entender por qué falló una request específica → usa **Traces**
- Ver el mensaje de error exacto de un usuario → usa **Logs**
- Rastrear el ID de una transacción específica → usa **Traces + Logs**

Note

Las métricas son tu **primera línea de defensa**. Te dicen *que algo está mal*. Los traces y logs te dicen *por qué*.

Logs — ¿Para qué sirven?

Logs: Registros de eventos discretos con contexto detallado. Ricos en información, costosos de almacenar y consultar.

✓ Úsalos para:

- Depurar errores específicos de un usuario
- Auditoría y compliance regulatorio
- Entender la secuencia exacta de eventos
- Análisis forense post-incidente

✗ No los uses para:

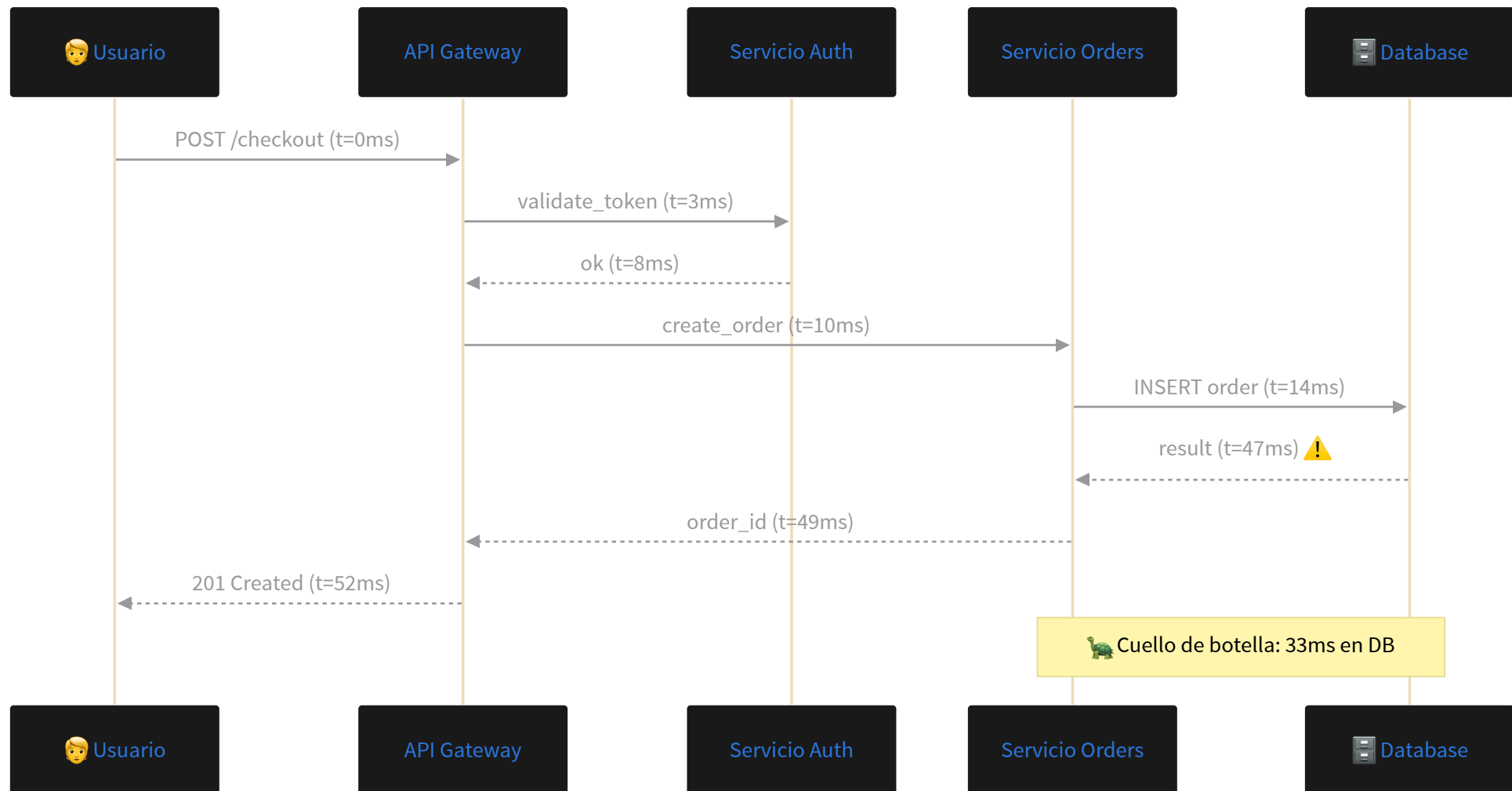
- Medir latencia o throughput → usa **Métricas (Histogram)**
- Rastrear el flujo entre microservicios → usa **Traces**
- Monitorear performance en tiempo real → usa **Métricas (Counter/Gauge)**

Anti-patrón frecuente y costoso

Usar **grep** o queries en logs para calcular una tasa de errores en tiempo real.
Eso es **lento, caro y no escala**. Para eso existe un Counter en tu sistema de métricas.

Traces — ¿Para qué sirven?

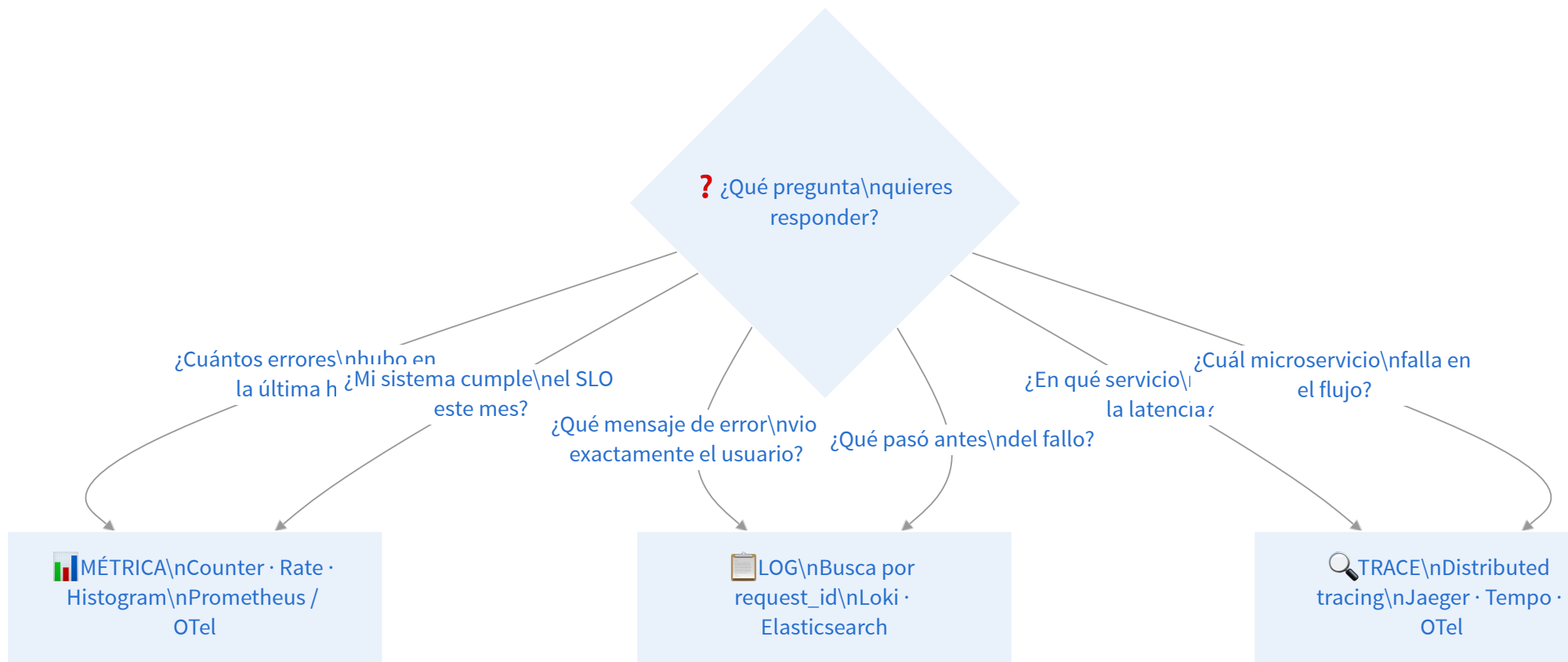
Traces (Trazas distribuidas): Seguimiento del camino completo de una request a través de múltiples servicios.



Tip

Sin traces, ves que el checkout tarda 52ms. **Con traces, ves exactamente que 33ms los consume la DB.** Esa es la diferencia entre observabilidad y monitoreo tradicional.

¿Cuándo usar qué? — Guía de decisión



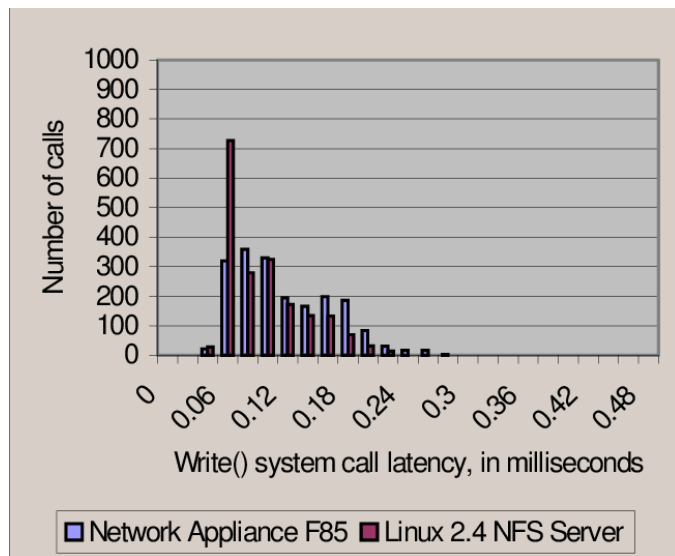
Metric Types

Las gráficas son solo representación visual

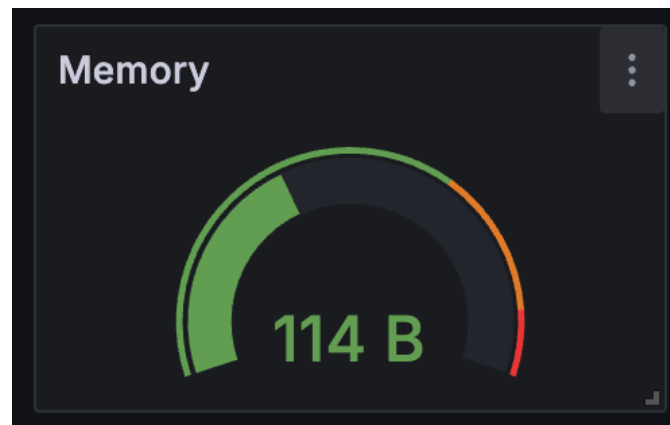
! Aclaración fundamental

Los tipos de métricas NO son tipos de gráficas.

Un Gauge puede mostrarse como línea, barra, número o velocímetro.
Lo que importa es la **semántica del dato**, no cómo se visualiza.



(a) Counter: siempre sube



(b) Gauge: sube y baja



Figure 1: Las gráficas son representaciones. Los tipos son Counter, Gauge e Histogram.

Los 3 Tipos de Métricas

Histogram — Distribución

Agrupa mediciones\nen buckets
(rangos)

✓ Latencia de requests\n✓
Tamaño de respuestas\n✓
Duración de jobs

Consulta: percentiles\np50, p95,
p99

Gauge — Valor instantáneo

Sube y baja libremente\ncomo
un termómetro

✓ CPU actual\n✓ Memoria
usada\n✓ Conexiones activas

Consulta: valor actual\nno
promedio en el tiempo

Counter — Contador acumulativo

Solo incrementa\n(o se resetea al
reiniciar)

✓ requests totales\n✓ errores
totales\n✓ bytes transmitidos

Consulta: rate(counter[5m])\n→
valor por segundo

Counter — Definición y uso

Counter: Valor que solo puede incrementar. Se resetea únicamente al reiniciar el proceso.

```
1 # Instrumentación con OpenTelemetry
2 from opentelemetry import metrics
3
4 meter = metrics.get_meter("mi-servicio")
5
6 # Definición del counter
7 http_requests = meter.create_counter(
8     name="http_requests_total",
9     description="Total de requests HTTP recibidas",
10    unit="1"
11 )
12
13 # Uso en el código
14 http_requests.add(1, {"method": "GET", "status": "200", "route": "/api/orders"})
15 http_requests.add(1, {"method": "GET", "status": "500", "route": "/api/orders"})
```

Consulta en Prometheus:

```
1 # Tasa de requests por segundo en los últimos 5 minutos
2 rate(http_requests_total{status=~"5.."}[5m])
```

Tip

Un counter solo tiene valor cuando calculas su **tasa de cambio** con `rate()`. El valor absoluto de un counter acumulado rara vez es útil por sí solo.

Gauge — Definición y uso

Gauge: Fotografía de un valor en un momento específico. Puede subir o bajar libremente.

```
1 # Instrumentación con OpenTelemetry
2 from opentelemetry import metrics
3
4 meter = metrics.get_meter("mi-servicio")
5
6 # Definición del gauge
7 active_connections = meter.create_observable_gauge(
8     name="db_active_connections",
9     description="Conexiones activas a la base de datos",
10    unit="1",
11    callbacks=[get_active_connections] # función que retorna el valor actual
12 )
13
14 memory_gauge = meter.create_observable_gauge(
15     name="process_memory_bytes",
16     description="Memoria usada por el proceso",
17     unit="By"
18 )
```

Ejemplos de uso:

Métrica	Tipo	¿Por qué Gauge?
<code>cpu_usage_percent</code>	Gauge	Puede bajar cuando baja la carga
<code>memory_used_bytes</code>	Gauge	Sube y baja con el GC
<code>active_db_connections</code>	Gauge	Refleja el estado actual del pool
<code>queue_depth</code>	Gauge	Los mensajes entran y salen

Histogram — Definición y uso

Histogram: Distribución de mediciones en rangos (buckets). Permite calcular percentil aproximados.

```
1 # Instrumentación con OpenTelemetry
2 from opentelemetry import metrics
3 import time
4
5 meter = metrics.get_meter("mi-servicio")
6
7 request_duration = meter.create_histogram(
8     name="http_request_duration_seconds",
9     description="Duración de requests HTTP en segundos",
10    unit="s"
11 )
12
13 # Uso: medir duración de cada request
14 start = time.time()
15 process_request()
16 duration = time.time() - start
17
18 request_duration.record(duration, {"method": "POST", "route": "/api/checkout"})
```

Consulta de percentiles en Prometheus:

```
1 # p99 de latencia en los últimos 5 minutos
2 histogram_quantile(0.99,
3   rate(http_request_duration_seconds_bucket[5m])
4 )
```

Usa percentiles, no promedios

Un **promedio de latencia de 100ms** puede esconder que el **p99 es 2 segundos**. Eso significa que 1 de cada 100 usuarios espera 2 segundos. El promedio oculta el problema. El p99 lo revela.

Cuando usar cada tipo — Resumen

Pregunta	Tipo	Ejemplo
¿Cuántas requests recibí?	Counter	<code>http_requests_total</code>
¿A qué tasa fallan las requests?	Counter + <code>rate()</code>	<code>rate(errors_total[5m])</code>
¿Cuánta CPU usa mi proceso ahora?	Gauge	<code>process_cpu_usage</code>
¿Cuántos items hay en la cola?	Gauge	<code>queue_depth</code>
¿Cuál es la latencia p99?	Histogram	<code>histogram_quantile(0.9 ...)</code>
¿Qué tamaño tienen las respuestas?	Histogram	<code>response_size_bytes</code>
¿Cuántos usuarios activos hay?	Gauge	<code>active_users</code>
¿Cuántos errores por minuto?	Counter + <code>rate()</code>	<code>rate(errors_total[1m]) 60</code>

The Four Golden Signals

Definidos por Google SRE. Si solo puedes medir cuatro cosas en tu servicio, mide estas.

Los 4 Golden Signals

★ Four Golden Signals —
Google SRE

 Latency\n¿Cuánto tarda\ntu sistema en responder?

SLO típico: p99 < 500ms

 SLO Latencia

 Traffic\n¿Cuánta demanda\nrecibe tu sistema?

baseline para capacity

 Capacity Planning

 Errors\n¿Qué fracción de\nrequests falla?

SLO típico: error rate < 0.1%

 SLO Disponibilidad

 Saturation\n¿Qué tan lleno\nestá tu sistema?

umbral: CPU < 80%

 SLO Saturación

Latency

El tiempo que tarda tu sistema en responder a una solicitud.

Important

Distingue siempre entre latencia de requests **exitosas** y **fallidas**.

Una request que falla en 1ms puede distorsionar tu p99 hacia abajo, ocultando un problema de latencia real en las request exitosas.

Métrica recomendada

`http_request_duration_seconds`

Tipo

Histogram

Query ejemplo

```
histogram_quantile(0.99,  
rate(http_request_duration_  
[5m]))
```

SLOs típicos: - `p50 < 100ms` — Usuario típico - `p95 < 300ms` — Mayoría de usuarios - `< 500ms` — Usuarios en el peor caso

Traffic

La demanda que ejerce sobre tu sistema.

Define el **baseline normal** sin el cual no puedes distinguir una anomalía de un pico esperado.

Sistema	Métrica de Traffic
API REST	Requests/segundo (RPS)
Streaming	Mensajes/segundo
Base de datos	Transacciones/segundo
Storage	Bytes escritos/leídos por segundo
Microservicio	Llamadas RPC/segundo

```
1 # RPS en tiempo real
2 rate(http_requests_total[1m])
3
4 # Comparar con la semana pasada
5 rate(http_requests_total[1m])
6 /
7 rate(http_requests_total[1m] offset 1w)
```

Errors

El porcentaje de requests que fallan.

Tres categorías de errores:

Explícitos

Fallos reconocidos por el protocolo

- HTTP 5xx
- Excepciones no controladas
- Timeouts del servidor

Implícitos

Fallos que parecen exitosos

- HTTP 200 con JSON malformado
- Respuesta vacía cuando se esperan datos
- Código de negocio erróneo en 200

Por política

Éxitos que violan el SLO

- HTTP 200 pero tardó cuando el SLO es 500ms
- Respuesta correcta pero fuera del Error Budget

```
1 # Error rate (errores explícitos)
2 sum(rate(http_requests_total{status=~"5.."}[5m]))
3 /
4 sum(rate(http_requests_total[5m]))
```

Saturation

¿Qué tan “lleno” está tu servicio?

La saturación predice problemas **antes** de que ocurran.

Recursos a monitorear:

- CPU usage por core
- Memoria usada vs disponible
- Connection pool de la DB
- Threads del application server
- Profundidad de la message queue
- Espacio en disco (tasa de crecimiento)

Regla de thumb

Un recurso al **70% de capacidad bajo carga normal** es una señal de alerta.

Un recurso al **90% de capacidad** es una emergencia inminente.

La latencia sube exponencialmente cerca de la saturación (queueing theory).

OpenTelemetry

¿Por qué instrumentación agnóstica?

El problema con los SDKs propietarios

Si instrumentas tu código con el SDK de Datadog directamente y decides migrar a otro vendor (por precio, por features, por lo que sea), tienes que **re-instrumentar toda tu aplicación**.

Eso puede significar semanas de trabajo y riesgo de regresiones.

La solución: separar instrumentación de exportación.

Mañana (sin re-instrumentar)

solo cambias el exporter

 Grafana Stack

solo cambias el exporter

 NewRelic

solo cambias el exporter

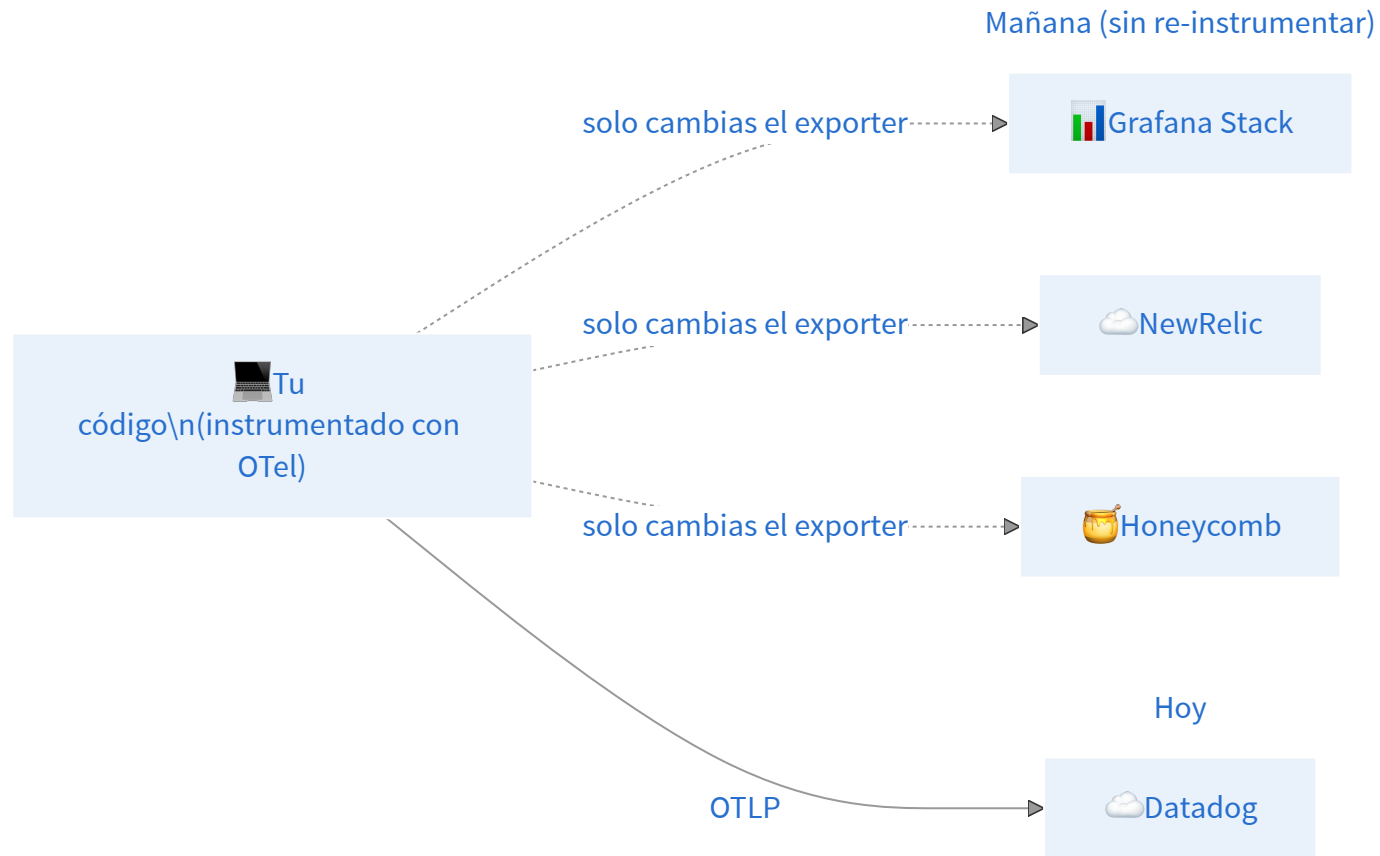
 Honeycomb

Hoy

OTLP

 Datadog

 Tu
código\n(instrumentado con
OTel)



¿Qué es OpenTelemetry?

OpenTelemetry (OTel) es un framework open source y vendor-neutral para instrumentar, generar, recolectar y exportar telemetría.

Es el **estándar de la industria**, parte de la Cloud Native Computing Foundation (CNCF)

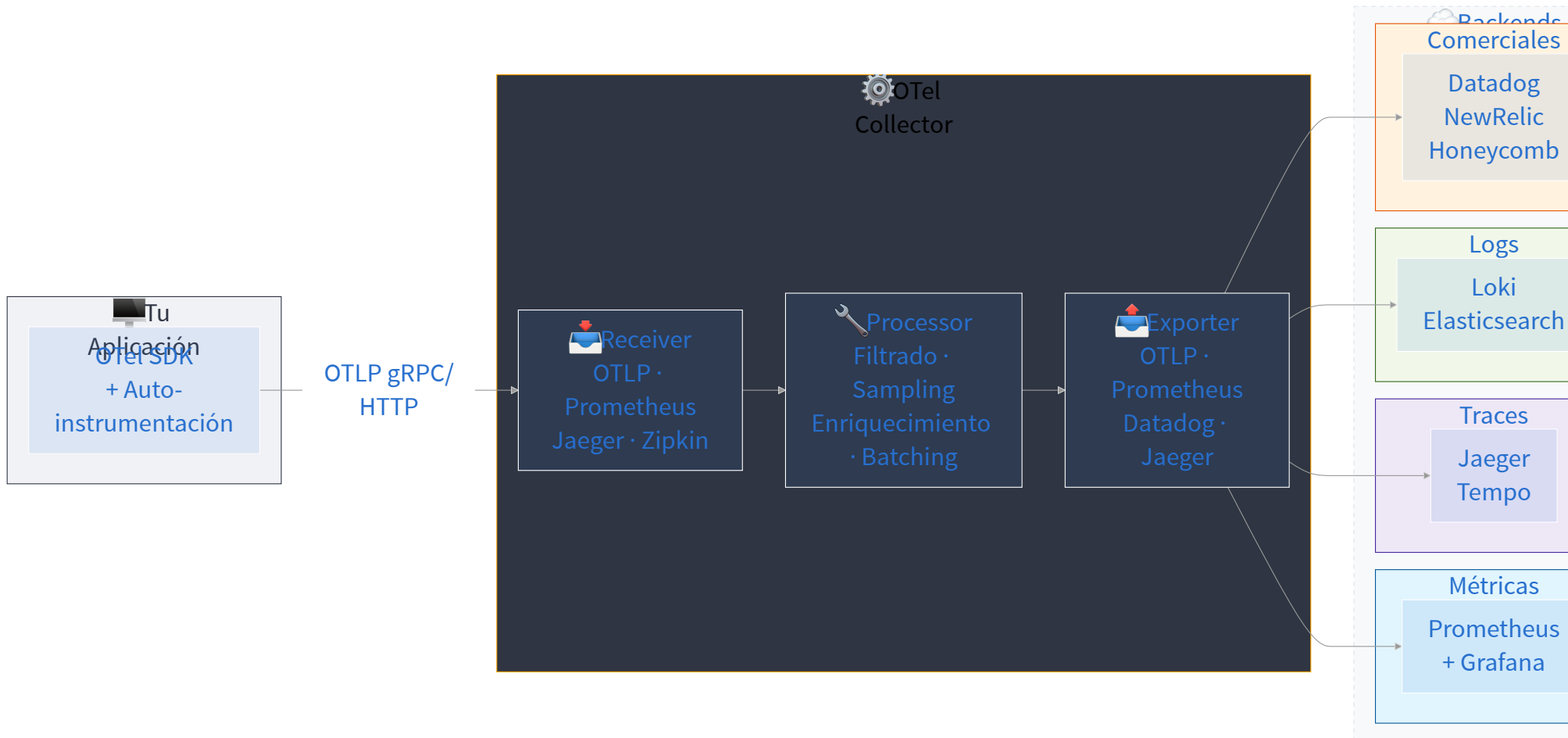
Qué incluye:

- APIs estandarizadas para los 3 pilares
- SDKs en 11+ lenguajes
- OTel Collector (pipeline de telemetría)
- Protocolo OTLP (OpenTelemetry Protocol)
- Instrumentación automática para frameworks populares

Lenguajes soportados:

-  Python, Java, Go
-  JavaScript/Node.js, .NET
-  Ruby, PHP, Rust
-  C++, Swift, Erlang
-  Y más en desarrollo...

Arquitectura de OpenTelemetry



OTel vs SDKs Proprietarios

Aspecto	 SDK Propietario (ej: Datadog)	 OpenTelemetry
Portabilidad	Lock-in al vendor	Cambia backend sin tocar código
Estándar	API propietaria	Estándar CNCF adoptado por la industria
Vendor	Uno solo	Cualquier backend compatible
Costo de migración	Re-instrumentar todo	Solo cambiar el exporter
Multi-señal	Depende del vendor	Métricas + Logs + Traces unificados
Open Source	Parcialmente	100% open source
Correlación	Dentro del mismo vendor	Cross-vendor, cross-tool

Regla de oro

Nunca importes el SDK de un vendor directamente en tu código de negocio.

Instrumenta con OTel → configura el exporter en la capa de infraestructura (Helm chart, variables de entorno, OTel Collector)

OTel — Ejemplo de instrumentación

```
1 #  INCORRECTO: Lock-in al vendor
2 import ddtrace
3 from ddtrace import tracer
4
5 ddtrace.patch_all()
6
7 @tracer.wrap("procesar_orden")
8 def procesar_orden(orden_id):
9     # Si migramos a NewRelic,
10    # reescribimos todo esto
11    return db.query(orden_id)
```

```
1 #  CORRECTO: Agnóstico con OTel
2 from opentelemetry import trace, metrics
3
4 tracer = trace.get_tracer("checkout-service")
5 meter = metrics.get_meter("checkout-service")
6
7 # Métricas
8 orders_counter = meter.create_counter(
9     "orders_processed_total"
10 )
11
12 def procesar_orden(orden_id: str):
13     with tracer.start_as_current_span("procesar_orden") as span:
14         span.set_attribute("orden.id", orden_id)
15
16         result = db.query(orden_id)
17
18         orders_counter.add(1, {
19             "status": "success",
20             "region": "us-east-1"
21         })
22     return result
23
24 # El backend se configura en variables de entorno:
25 # OTEL_EXPORTER_OTLP_ENDPOINT=http://collector:4317
```

Backends compatibles con OTel

Categoría	Open Source	Comerciales
Métricas	Prometheus, Grafana Mimir, VictoriaMetrics	Datadog, NewRelic
Traces	Jaeger, Zipkin, Grafana Tempo	Datadog APM, Honeycomb, Lightstep
Logs	Grafana Loki, Elasticsearch/OpenSearch	Datadog Logs, NewRelic Log
All-in-one	Grafana Stack (Loki+Tempo+Mimir)	Datadog, NewRelic, Honeycomb

Note

OTel **no reemplaza** a Datadog o NewRelic. Define **cómo instrumentas tu código**.

Datadog y NewRelic son solo dos de los muchos posibles destinos de tus datos.

Puedes enviar al mismo tiempo a Prometheus (gratis) y Datadog (de pago) cambiando solo la configuración del Collector.

**Elegir las Métricas
Correctas**

Proceso de selección — Siempre desde los SLOs

1 Define tu SLO\n¿Qué garantizas al usuario?\nEj: p99 latencia < 500ms



2 Identifica el SLI\n¿Qué métrica lo mide?\nEj: http_request_duration_seconds



3 Elige el tipo de métrica\nCounter · Gauge · Histogram



4 Define etiquetas (labels)\nservicio, entorno, región, versión



5 ¿Qué acción requiere?

Características de una Buena Métrica

Una buena métrica debe ser:

✓ Entendible

Clara para todo el equipo, no solo para quien la creó.

¿Puede un on-call de guardia entenderla a las 3am?

✓ Accionable

Cuando cambia, alguien sabe exactamente qué hacer.

Si no sabes qué hacer cuando sube, no es una buena métrica para alertar.

✓ Mejorable

Puede optimizarse con el tiempo.

¿Hay algo que el equipo pueda hacer para moverla en la dirección correcta?

✓ Multidimensional

Con etiquetas que permitan filtrar, agrupar y aislar problemas.

¿Puedo saber si el problema es solo en producción o en todos los entornos?

Multidimensional — El poder de las etiquetas

El etiquetado (labeling) añade **dimensiones** a tus métricas para aislar problemas rápidamente.

```
1 # Sin etiquetas: ¿qué servicio falla? ¿en qué entorno?  
2 http_errors_total = 342 ← inútil para diagnóstico  
3  
4 # Con etiquetas: diagnóstico inmediato  
5 http_errors_total{  
6   service="checkout",      # ← ¿qué servicio?  
7   env="production",        # ← ¿qué entorno?  
8   region="us-east-1",      # ← ¿qué región?  
9   version="v2.3.1",        # ← ¿qué versión deployé?  
10  status="503"              # ← ¿qué tipo de error?  
11 } = 342
```

Etiquetas recomendadas como baseline:

Etiqueta	Valores ejemplo	Para qué sirve
<code>env</code>	production, staging, dev	Separar entornos
<code>service</code>	checkout, payments, auth	Identificar componente
<code>region</code>	us-east-1, eu-west-1	Aislar problemas geográficos

Etiqueta	Valores ejemplo	Para qué sirve
version	v2.3.1, v2.4.0	Correlacionar con deploys

Problema de Cardinalidad

Nunca uses como etiqueta valores de alta cardinalidad como `user_id`, `request_id` o `session_id`. Eso puede crear millones de series de tiempo y destruir el rendimiento de tu sistema de métricas. Para datos de alta cardinalidad, usa **Traces y Logs**.

Alerting & Alerting Fatigue

El problema del Alerting Fatigue

Principio crítico

Una alerta que no requiere una acción inmediata NO es una alerta.
Es una notificación, y debe eliminarse o moverse a un canal asíncrono para evitar el agotamiento del equipo.

 200 alertas/día\nsin
contexto ni prioridad

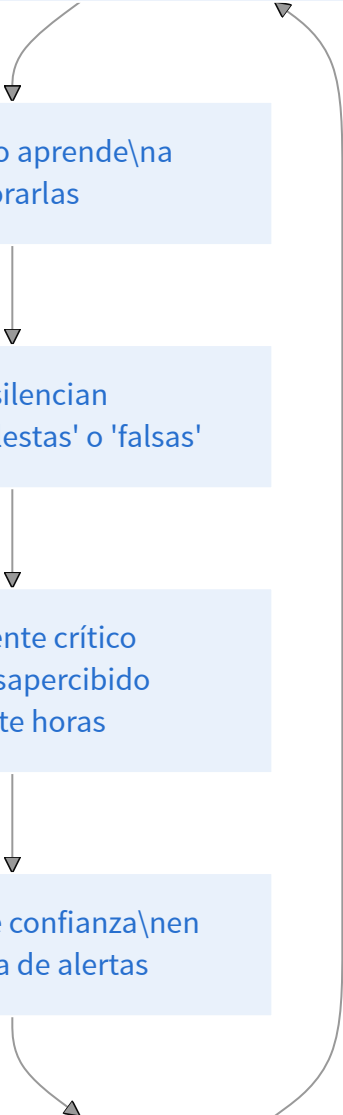
 El equipo aprende\na
ignorarlas

 Se silencian
alertas\n'n'molestas' o 'falsas'

 Incidente crítico
pasa\nndesapercibido
durante horas

 Pérdida de confianza\nnen
el sistema de alertas

 Burnout del equipo\nnde
on-call



Urgente

¿Requiere atención en los próximos **minutos**?

No en horas, no mañana.

Ahora.

Accionable

¿Hay algo **concreto** que hacer en este momento?

¿Existe un runbook?

¿Alguien sabe qué hacer?



Impacta al usuario

¿Está afectando o afectará el SLO comprometido?

¿Hay usuarios afectados ahora mismo o en los próximos minutos?

Alerta vs Notificación — La distinción clave

Dimensión	 Alerta	 Notificación
Canal	PagerDuty · Llamada · SMS	Slack · Email · Ticket · Dashboard
Tiempo de respuesta esperado	Inmediato (< 5 minutos)	Próximas horas o días
¿Despierta a alguien?	Sí, incluso a las 3am	No
Ejemplo válido	Error rate supera el SLO en prod	Disco al 65% (tendencia de 2 semanas)
Si no es accionable	Eliminar	Revisar en el próximo sprint
Requiere runbook	 Siempre	 Opcional

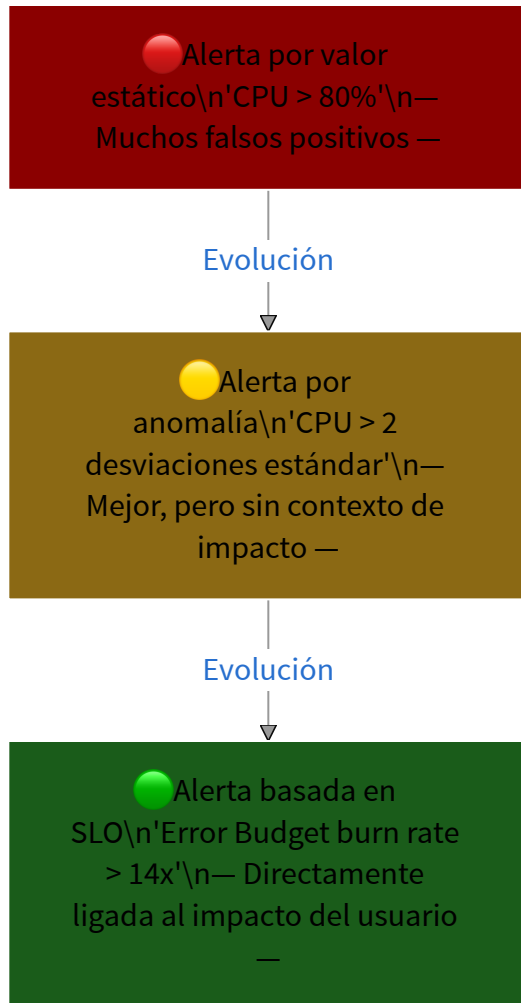
Ejercicio práctico para tu equipo

Revisa cada alerta activa y hazte esta pregunta:

“¿Qué hago exactamente cuando recibo esta alerta a las 2am?”

Si la respuesta es *“la ignoro”* o *“no sé”* → esa alerta debe eliminarse hoy.

Tipos de Alertas — De peor a mejor



Alerta basada en Error Budget (gold standard — Google SRE):

- 1 # Esta alerta significa: "Estás consumiendo tu error budget 14x más rápido de lo normal"
- 2 # Si continúa así, en ~2 horas habrás consumido el budget de todo el mes
- 3 - alert: [SLOErrorBudgetBurnRateCritical](#)

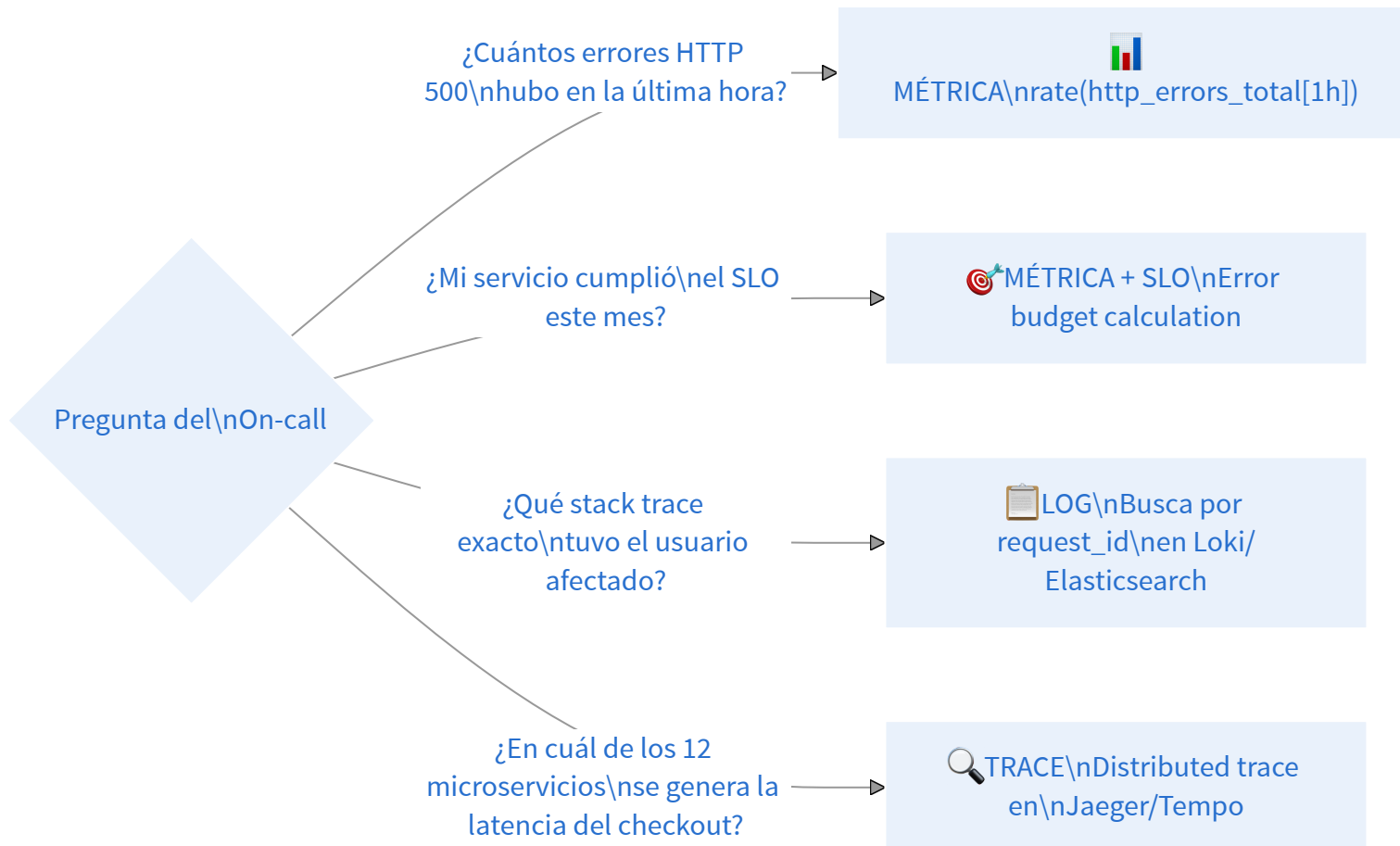
```
4  expr: |
5    (
6      rate(http_requests_total{status=~"5.."}[1h])
7      / rate(http_requests_total[1h])
8    ) > (14 * 0.001)
9  for: 5m
10 labels:
11   severity: critical
12   team: platform
13 annotations:
14   summary: "SLO en riesgo: Error budget burning 14x faster than normal"
15   description: "Service {{ $labels.service }} error rate: {{ $value | humanizePercentage }}"
16   runbook: "https://wiki.empresa.com/runbooks/high-error-budget-burn"
17   dashboard: "https://grafana.empresa.com/d/slo-dashboard"
```

Tip

Toda alerta debe tener un **runbook asociado.** Si no puedes escribir los pasos del runbook, no deberías crear la alerta. El runbook es la prueba de que la alerta es accionable.

Logs & Best Practices

Cuándo usar Logs vs Métricas vs Traces



Best Practices — Logging

✓ Hacer:

- Establecer objetivos claros de logging
- Usar log levels correctamente
- Estructurar logs en JSON
- Escribir entradas significativas con contexto
- Usar canonical log lines por request
- Agregar y centralizar logs
- Establecer políticas de retención
- Proteger y auditar acceso a logs

✗ Evitar:

- Loguear datos sensibles (PII, passwords, tokens)
- Ignorar el costo de performance del logging
- **Usar logs para monitorear performance**
→ usa métricas
- Log levels incorrectos (todo en ERROR, todo en DEBUG)
- Logs no estructurados difíciles de parsear
- Sin correlación entre requests (falta de request_id)

Structured Logging — Ejemplo práctico

```
1 # ❌ Log NO estructurado
2 # Difícil de parsear, buscar y correlacionar
3
4 ERROR 2024-01-15 14:23:01 - User 12345
5 failed to checkout order 98765
6 - upstream timeout after 30000ms
7 from payments service
```

```
1 // ✅ Log ESTRUCTURADO (JSON)
2 // Parseable, buscable, correlacionable con trace_id
3 {
4   "timestamp": "2024-01-15T14:23:01.234Z",
5   "level": "ERROR",
6   "service": "checkout-api",
7   "version": "v2.3.1",
8   "env": "production",
9   "region": "us-east-1",
10  "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
11  "span_id": "00f067aa0ba902b7",
12  "request_id": "req_7f3a9b2c",
13  "user_id": "usr_12345",
14  "order_id": "ord_98765",
15  "error_type": "upstream_timeout",
16  "upstream_service": "payments-api",
17  "upstream_url": "http://payments:8080/charge",
18  "duration_ms": 30042,
19  "message": "Checkout failed: payment service t",
20 }
```

Tip

El campo `trace_id` en el log permite saltar directamente al trace distribuido en Jaeger/Tempo desde el log en Loki. Eso es **correlación entre pilares** y es lo que hace poderosa la observabilidad real.

Log Levels — Guía de uso correcto

Level	Cuándo usar	Ejemplo
DEBUG	Información detallada para desarrollo. Nunca en prod por defecto	"Calculando descuento para usuario X con regla Y"
INFO	Eventos normales del flujo de negocio	"Orden procesada exitosamente: order_id=98765"
WARN	Situación inesperada pero el sistema sigue funcionando	"Reintentos: 2/3 a conectar a payment api"
ERROR	Fallo en una operación específica. Requiere revisión	"Fallo al procesar orden: timeout en payments"
FATAL	Error irreparable. El proceso no puede continuar	"No se puede conectar a la DB en

Level

Cuándo usar

Ejemplo

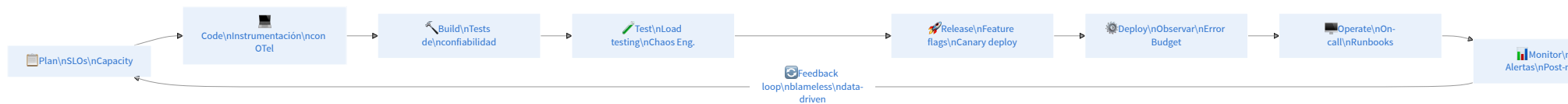
startup. Abortando

Warning

Anti-patrón: Loguear todo como **ERROR** para “no perderse nada”. Eso produce la misma pérdida de señal que el Alerting
Fatigue: cuando todo es error, nada es error.

DevOps & Continuous Improvement

Monitoreo en el Ciclo DevOps



La observabilidad **cierra el loop**. Sin ella, el feedback de producción tarda semanas en llegar al equipo de desarrollo. Con ella, tarda minutos.

Beneficios de una Estrategia de Observabilidad Madura

Para el equipo técnico:

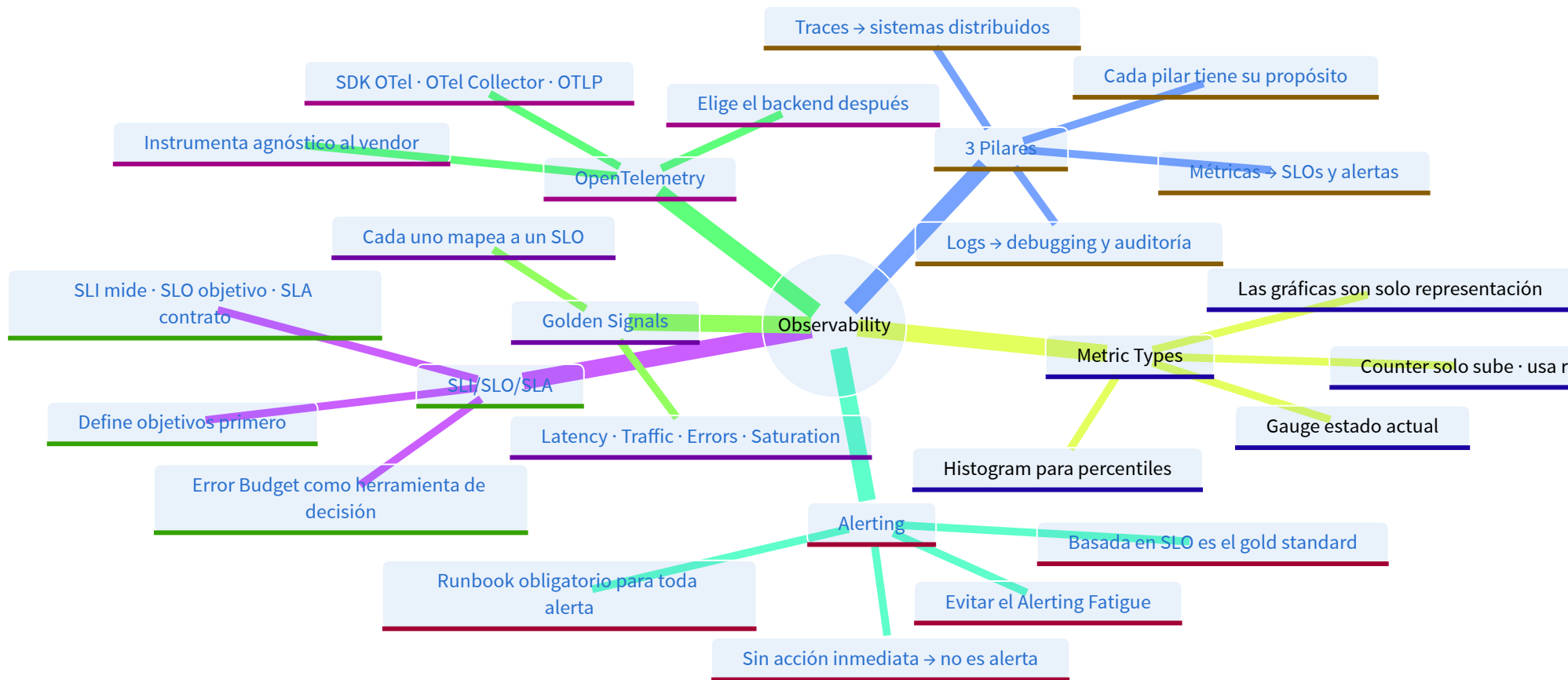
-  **MTTR reducido:** Localizar el problema en minutos, no horas
-  **Deploys con confianza:** El Error Budget dice cuándo es seguro
-  **Cultura blameless:** Los datos reemplazan las suposiciones
-  **Capacity planning:** Las tendencias predicen cuándo escalar

Para el negocio:

-  **Reducción de incidentes:** Detección proactiva antes del impacto
-  **Acuerdos claros:** SLAs basados en datos reales, no estimaciones
-  **Colaboración Dev+Ops:** El mismo lenguaje: SLOs y datos
-  **Mejora continua:** Cada incidente genera aprendizaje medible

Key Takeaways

Mapa mental de la sesión



Resumen — Lo que cambia desde hoy

Antes

Monitorear todo
sin objetivo

“Tipos de
métricas” =
tipos de gráficas

Logs para todo,
incluyendo
performance

SDK propietario
de un vendor en
el código

200 alertas/día
que nadie lee

alert: CPU >
80%



Después de esta sesión

Definir SLOs primero, luego los SLIs

Counters · Gauges · Histograms tienen semántica propia

Métricas para performance, logs para debugging, traces para flujos

OpenTelemetry agnóstico + exporter configurable

Solo alertas urgentes + accionables + con runbook

alert: Error budget burn rate > 14x

Antes

Grep en logs
para calcular
error rates

`user_id` como
label de métrica



Después de esta sesión

Counter con `rate()` en Prometheus

`user_id` en traces y logs (alta cardinalidad)

Recursos recomendados

-  **Google SRE Book** (gratuito): sre.google/sre-book — Capítulos de SLOs y Alerting
-  **OpenTelemetry Docs**: opentelemetry.io/docs
-  **Prometheus Best Practices**: prometheus.io/docs/practices
-  **“Observability Engineering”** — Charity Majors, Liz Fong-Jones (O’Reilly)

Clase Siguiente



Próimos temas...